

Lecture # 13

Android Services

By

Dr. Muhammad Bilal

Services

- A Service is an application component that can perform long-running operations in the background.
- It does not provide a user interface
- Once started, a service might continue running for some time, even after the user switches to another application.
- Additionally, a component can bind to a service to interact with it.
- For example, a service can handle network transactions, play music, perform file I/O, or interact with a content provider, all from the background.

Types of Services

- There are three types of services
 - Foreground
 - Background
 - Bound

Foreground Service

- A foreground service performs some operation that is noticeable to the user.
- For example, an audio app would use a foreground service to play an audio track.
- Foreground services must display a Notification
- Foreground services continue running even when the user isn't interacting with the app.
- When you use a foreground service, you must display a notification so that users are actively aware that the service is running.
- This notification cannot be dismissed unless the service is either stopped or removed from the foreground.

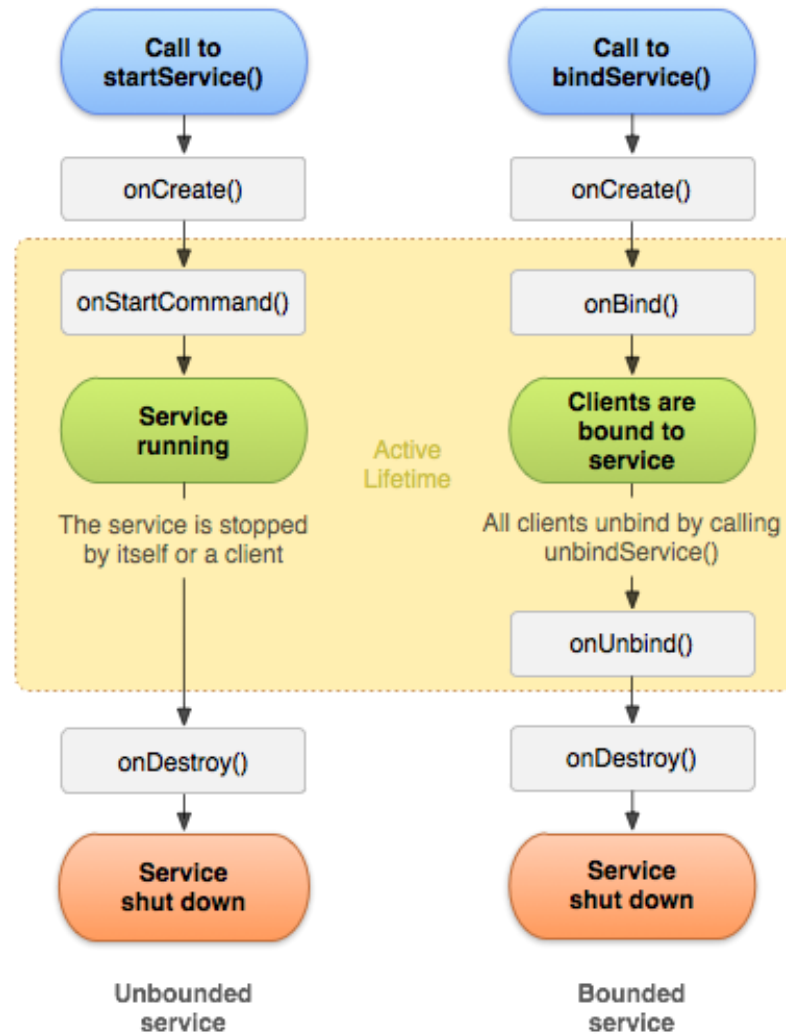
Background Service

- A background service performs an operation that isn't directly noticed by the user.
- For example, if an app used a service to compact its storage, that would usually be a background service.

Bounded Service

- A service is bound when an application component binds to it by calling `bindService()`
- A bound service offers a client-server interface that allows components to interact with the service, send requests, receive results
- A bound service runs only as long as another application component is bound to it.
- Multiple components can bind to the service at once, but when all of them unbind, the service is destroyed.

Service Lifecycle



onCreate()

- The system invokes this method to perform one-time setup procedures when the service is initially created (before it calls either **onStartCommand()** or **onBind()**).
- If the service is already running, this method is not called.

onStartCommand(Intent intent, int flags, int startId)

- The system invokes this method by calling **startService()** when another component (such as an activity) requests that the service be started.
- When this method executes, the service is started and can run in the background indefinitely.
- If you implement this, it is your responsibility to stop the service when its work is complete by calling **stopSelf()** or **stopService()**.
- If you only want to provide binding, you don't need to implement this method.

onStartCommand() parameters

Parameters	
intent	Intent: The Intent supplied to Context.startService(Intent) , as given.
flags	int: Additional data about this start request. Value is either 0 or a combination of START_FLAG_REDELIVERY , and START_FLAG_RETRY
startId	int: A unique integer representing this specific request to start. Use with stopSelfResult(int) .
int	The return value indicates what semantics the system should use for the service's current started state Values are either START_STICKY , or START_NOT_STICKY

onBind()

- The system invokes this method by calling **bindService()** when another component wants to bind with the service (such as to perform RPC).
- In your implementation of this method, you must provide an interface that clients use to communicate with the service by returning an **IBinder**.
- You must always implement this method; however, if you don't want to allow binding, you should return null.

onDestroy()

- The system invokes this method when the service is no longer used and is being destroyed.
- Your service should implement this to clean up any resources such as threads, registered listeners, or receivers.
- This is the last call that the service receives.

Service Class

- The Service class is the base class for all services.
- When you extend this class, it's important to create a new thread in which the service can complete all of its work; the service uses your application's main thread by default, which can slow the performance of any activity that your application is running.

```
public class MyService extends Service {
```

Declaring Service in the Manifest

- You must declare all services in your application's manifest file, just as you do for activities and other components.
- To declare your service, add a **<service>** element as a child of the **<application>** element. Here is an example:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myserviceapp">

    <application
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service android:name=".MyService"/>
    </application>

</manifest>
```

Starting and Stopping of a Service

- You can start a service from an activity or other application component by passing an **Intent** to **startService()**

```
intent = new Intent(this, MyService.class);  
startService(intent);
```

- The service can be stopped using stopService(), from where the calling activity

```
intent = new Intent(this, MyService.class);  
stopService(intent);
```

- To stop service with in the service use selfStop() or selfStop(serviceID)

```
stopSelf(startID);
```

MainActivity.java

```
public class MainActivity extends AppCompatActivity {  
  
    Intent serviceIntent;  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main_service);  
    }  
  
    public void startService(View view){  
        serviceIntent = new Intent(getApplicationContext(), MyService.class);  
        startService(serviceIntent);  
    }  
  
    public void stopService(View view){  
        serviceIntent = new Intent(getApplicationContext(), MyService.class);  
        stopService(serviceIntent);  
    }  
}
```

Threads

- A thread is a thread of execution in a program
- The Java Virtual Machine allows an application to have multiple threads of execution running concurrently
- Thread class can be create using Thread API

```
class serviceThread extends Thread
```

- The run() method must be implemented to define what should be run in the thread

```
@Override  
public void run() {  
  
}
```

- The thread can be start using

```
serviceThread thread = new serviceThread(startId);  
thread.start();
```

serviceThread Class

```
class serviceThread extends Thread {
    int threadID;

    serviceThread(int threadID){
        this.threadID = threadID;
    }

    @Override
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                Log.d("MyService", "serviceRunnable Running i = " + i);
                Thread.sleep(1000);
            }
        } catch (Exception e) {
            Log.d("MyService", "serviceRunnable Error " + e);
        }
        stopSelf(threadID);
    }
}
```

Implement onStartCommand()

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    Log.d("MyService", "onStartCommand: Service Started");

    serviceThread thread = new serviceThread(startId);
    thread.start();

    return START_STICKY;
}
```

MyService.java

```
public class MyService extends Service {
    @Nullable
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        Log.d("MyService", "onStartCommand: Service Started");

        serviceThread thread = new serviceThread(startId);
        thread.start();

        return START_STICKY;
    }

    class serviceThread extends Thread {
        int threadID;

        serviceThread(int threadID) {
            this.threadID = threadID;
        }

        @Override
        public void run() {
            try {
                for (int i = 0; i < 10; i++) {
                    Log.d("MyService", "serviceThread Running i = " + i);
                    Thread.sleep(1000);
                }
            } catch (Exception e) {
                Log.d("MyService", "serviceThread Error " + e);
            }
            stopSelf(threadID);
        }
    }
}
```